AI Agent Skills and MCP: Building the Right AI Agent Pattern

Stephen Simon

Chapter 2: Building AI Agent Skills

Reusing the Skill Across Multiple Tasks

Use one launch-risk-review Skill for a go/no-go check, a vendor email, and a full review

So far, every request has been a weekly status review. But the know-how in the `launch-risk-review` Skill isn't about status updates. It's about judging risk to a launch. In this video you point the same Skill at three different tasks, with no changes to the Skill, and then check where its boundaries are.

Before you start

You need the `launch-risk-review` Skill from video 02_01, with the template unchanged. If you added the Milestone column in the previous video, undo it. To get to the right starting point, run this from the folder that contains `orbit-launch` and `solutions`:

```
node solutions/catch-up.mjs 02_01
```

Then start a new Claude Code session in `orbit-launch`.

What's already in the workspace

- `inputs/go-no-go-notes.md` contains notes from an October 3 meeting preparing a beta exit go/no-go decision.
- `inputs/vendor-email-paygate.md` contains an email from the payments vendor, PayGate, forwarded to the team by Priya with a short note.
- `inputs/status-update-week-40.md` contains the October 2 status update you reviewed in the previous videos.

These are three different kinds of documents, written by different people for different purposes.

Step 1: A go/no-go check

Send this request:

```
We're deciding whether to exit beta on October 16. Prepare a go/no-go risk
view from @inputs/go-no-go-notes.md
```

Look for:

- The Skill loads, and the risks come back in the same report format as the weekly reviews. You may also get a short go/no-go recommendation at the top.
- The pen test booking, rated at least **Medium** because nobody owns it.
- The duplicate-charge issue: four customers affected, oldest report from September 22. Expect **Medium**, with ticket priorities listed under **Missing information**.
- The load test, which can't start until the checkout integration is done.

Why isn't the duplicate-charge issue High? The rubric's systemic issue rule makes a cluster of tickets High only if they're P1 or P2. Meeting notes don't include priorities, so the agent applies the rule as far as the evidence allows and tells you what it would need to go further. Same rule, same behavior, different kind of document.

Step 2: The impact of a single email

Clear the conversation (`/clear` in Claude Code), then send:

```
How does @inputs/vendor-email-paygate.md affect the launch?
```

Here's the email, including Priya's note:

```
**Forwarded by:** Priya Nair, to the Orbit 2.0 launch team
**Date:** October 3, 2026

> Heads-up, team. With certification moving to November 2, we need about
three weeks after that to finish the checkout integration and run the load
test. That puts Payments v2 go-live in the first week of December, after
our November 16 launch date. Original email below.

---

**From:** Rachel Kim, Partner Manager, PayGate
**To:** Priya Nair, Orbit Labs
**Date:** October 3, 2026
**Subject:** Update on your sandbox certification

Hi Priya,

I wanted to give you a heads-up before our call next week. Our compliance
team is working through an audit backlog, and your sandbox certification
is now expected on **November 2** instead of October 12.

We can offer an expedited review for an additional fee. It usually saves
about a week, but I can't guarantee a date.

Production keys are issued two to three business days after certification.

Sorry for the change. Happy to talk through options.

Rachel
```

Look for:

- A **High** risk: Payments v2 go-live moves to the first week of December, more than two weeks after the November 16 launch.
- The date math shown in the evidence: certification on November 2, then about three weeks of integration and load testing.
- The expedited review offered by PayGate showing up as a recommended next action, without being treated as a guaranteed fix.

This is a one-document question, and the agent still applies the full process: find the dates, assess severity with the rubric, attach evidence, and use the template.

Step 3: Everything together

Clear the conversation again and send:

```
Do a full launch review using everything in @inputs/
```

Look for:

- **Overall status: Off track**, because there are at least two High risks: the PayGate slip and the pen test booking.
- **A conflict called out in the evidence.** The week 40 update says certification is "still expected October 12." The vendor email, sent a day later, says November 2. The Skill's rules say to prefer the most recent source and mention the conflict.
- **Duplicates combined.** The duplicate charges appear as "a couple of reports" in the status update and "four customers" in the meeting notes. They should be one risk, with evidence from both documents.

Try it yourself: check the boundary

This one isn't shown in the video. Clear the conversation and send a request that's related to the launch but isn't about risk:

```
Draft a short launch announcement for the Orbit 2.0 blog.
```

The agent should write the announcement without loading the Skill. That's correct. A Skill should be reused for the kind of work it describes, not for everything that mentions a launch.

Why this works

- **The Skill encodes a capability, not an input format.** Its process (find the dates, collect risks, apply the rubric, attach evidence, write the report) works on status updates, meeting notes, and emails alike.
- **The description names situations, not documents.** Reviewing a launch, checking readiness, preparing a go/no-go recommendation, and assessing how an update affects the launch date are all listed. That's why each request in this video triggered the Skill.

- **Some rules only pay off when tasks vary.** “Prefer the most recent source” and “combine duplicates” did nothing for a single status update. With three documents, they’re what keep the review accurate.
- **The boundaries are part of the design.** The description is specific enough that unrelated launch work, like writing a blog post, doesn’t pull the Skill in.

Designing for reuse

When you write your own Skills, a few habits make them reusable:

- Describe the capability, then list the situations people will actually bring to it, in their words.
- Write the process so it doesn’t assume one kind of input document.
- Add rules for what happens when sources disagree or overlap.
- Keep one capability per Skill. If you want help with launch announcements, that’s a separate Skill with its own description.
- Test the Skill on inputs it should handle and on requests it should ignore.